DAA MINI PROJECT · FINAL YEAR B.E.

Quick Sort vs Multithreaded Quick Sort

Your mini project explained section by section, with the results interpreted and probable viva / oral questions with answers

1	**The project at a glance**	Part 1
2	**Quick Sort and its analysis**	Part 2
3	**Multithreading, multiprocessing and the GIL**	Part 3
4	**Code walkthrough — quicksort.py**	Part 4
5	**Your results explained**	Part 5
6	**Probable viva questions with answers**	Part 6

Based on your report **DAA_Mini_Project_QuickSort_Report.docx** and its program **quicksort.py**.
How to use: read the theory once, trace the worked example by hand, then cover the answers and test yourself on the questions. Answers are written the way you would say them in a viva — short first line, then the detail if the examiner digs deeper.

PART 1

The project at a glance

Title: Quick Sort and Multithreaded Quick Sort — Running-Time Comparison and Best-Case / Worst-Case Analysis
Student: Yash Milind Dhadge · **Program:** `quicksort.py` (Python 3, standard library only)

Problem statement (in your own words)

Implement normal (sequential) Quick Sort and a multithreaded Quick Sort, compare how long each takes, and analyse each one's best and worst case. Your report goes further than the bare statement and does five things:

1. Sequential Quick Sort with a **first-element pivot**, tested on random, sorted, reverse-sorted and all-equal inputs from n = 10,000 to 1,000,000.
2. A **multithreaded** version that sorts the two partitions in separate threads, with a **size threshold** below which no new thread is created.
3. **Comparison counting** to prove the best case is O(n log n) and the worst case is O(n^2).
4. Repeating the runs with **random** and **median-of-three** pivots.
5. **Speedup vs number of threads/processes**, and why it stops growing — plus a **multiprocess** version, because Python threads cannot run CPU work in parallel.

Your results in one table (memorise this)

Question	Your answer, backed by your numbers
Best case	balanced partitions → Θ(n log n); measured comparisons **exactly** equal the recurrence (39,917 for n = 4000)
Worst case	sorted, reverse-sorted or all-equal input with first-element pivot → exactly n(n−1)/2 comparisons (7,998,000 for n = 4000, ~200× the best case)
Random input	≈ $2(n+1)H_n - 4n \approx 1.39\, n \log_2 n$; average over 200 runs 54,993 vs expected 54,989 for n = 4000
Threads vs sequential	**no speedup** (0.83× – 1.01×) because of Python's **GIL**
Processes vs sequential	real speedup ≈ **1.3×** on 2 cores for large n; **slower** (0.75×) for n = 10,000
Effect of pivot rule	random and median-of-three remove the sorted / reverse-sorted worst case; **all-equal stays O(n^2)** for every rule
Why speedup stops	only 2 cores, serial first partition, data copying → **Amdahl's law**

ONE-MINUTE ANSWER TO "EXPLAIN YOUR PROJECT"

"I implemented Quick Sort in Python in three forms — sequential, multithreaded and multiprocess — and compared their running times on inputs up to one million elements. I also counted comparisons to verify the theory: balanced partitions give n log n and sorted input with a first-element pivot gives exactly n(n−1)/2. Threads gave no speedup because of Python's Global Interpreter Lock, but separate processes gave about 1.3× on a 2-core machine for large inputs. Finally I showed that random and median-of-three pivots remove the sorted-input worst case, though all-equal keys still need three-way partitioning."

PART 2

Quick Sort and its analysis

How Quick Sort works

Quick Sort (C. A. R. Hoare, 1962) is **divide and conquer**:

1. **Divide** — choose a pivot and **partition**: smaller elements to its left, the rest to its right. The pivot is now in its **final sorted position**.
2. **Conquer** — sort the left and right parts recursively.
3. **Combine** — nothing; the array is sorted in place.

Your partition (Lomuto scheme, pivot at the front)

Your `partition()` first moves the chosen pivot to `a[lo]` (for the "first" rule nothing moves), then:

```
pivot = a[lo]; i = lo
for j = lo+1 .. hi:
    if a[j] < pivot:          # strictly less
        i += 1; swap a[i], a[j]
swap a[lo], a[i]              # pivot into its final place
return i
```

It makes **exactly hi – lo comparisons** — one per element other than the pivot. That is why `count_comparisons` simply adds `hi - lo` for every call, which makes the counts exact.

Trace on [4, 7, 2, 9, 1, 5, 8, 3], pivot = 4:

j	a[j]	< 4?	array after the step
1	7	no	[4, 7, 2, 9, 1, 5, 8, 3]
2	2	yes → i = 1, swap	[4, **2**, 7, 9, 1, 5, 8, 3]
3	9	no	—
4	1	yes → i = 2, swap	[4, 2, **1**, 9, 7, 5, 8, 3]
5, 6	5, 8	no	—
7	3	yes → i = 3, swap	[4, 2, 1, **3**, 7, 5, 8, 9]
end		swap a[0] ↔ a[3]	[3, 2, 1, **4**, 7, 5, 8, 9]

7 comparisons for 8 elements; 4 is in its final place at index 3.

Why `<` matters: equal keys are never moved to the left side. On an **all-equal** array every element stays on the right, so each partition removes only the pivot → n(n−1)/2 comparisons whatever the pivot rule. This is exactly what your Table 4 shows (1,999,000 for n = 2000 with all three rules).

Best, average and worst case

Case	When	Recurrence	Comparisons	Time
Best	pivot is always the median (n/2 : n/2)	$T(n) = 2T(n/2) + \Theta(n)$	$\approx n \log_2 n$	$\Theta(n \log n)$
Average	random order	—	$2(n+1)H_n - 4n \approx 1.39\, n \log_2 n$	$\Theta(n \log n)$
Worst	pivot always min or max (0 : n−1)	$T(n) = T(n-1) + \Theta(n)$	$n(n-1)/2$	$\Theta(n^2)$

Worst case derivation: (n−1) + (n−2) + … + 1 = n(n−1)/2. For n = 4000: 4000 × 3999 / 2 = 7,998,000 ✓. **Best case derivation:** $\log_2 n$ levels of recursion, about n comparisons per level → $n \log_2 n$. By the Master theorem: a = 2, b = 2, f(n) = n = n^($\log_2$ 2) → case 2 → Θ(n log n). **Space:** recursion stack O(log n) best/average, O(n) worst — which is why you raised the recursion limit to 50,000 and tested worst-case inputs only up to n = 4000.

$\mathbf{H_n}$ is the n-th harmonic number 1 + 1/2 + … + 1/n ≈ ln n + 0.577. For n = 1000: 2 × 1001 × 7.485 − 4000 ≈ **10,986** — the "theory" value in your table.

How you built a best-case input (`gen_best`)

For a first-element pivot, the best case is an arrangement where the first element of **every** sub-array is its median. You built it by running the partition **backwards**: if L and R are best-case arrangements of the smaller and larger values and p is the median, then `[p] + L[1:] + [L[0]] + R` is partitioned by p into exactly L, p, R.

Example for 1…7: `gen_best` gives **[4, 1, 3, 2, 6, 5, 7]**. Partition on 4 → [2, 1, 3] **4** [6, 5, 7]; then 2 is the median of [2, 1, 3] and 6 the median of [6, 5, 7]. Every split is perfect. Its exact comparison count follows C(n) = (n − 1) + C(⌊(n−1)/2⌋) + C(⌈(n−1)/2⌉) — your `theory_best` function, which matched the measured counts exactly (7,987 / 17,964 / 39,917).

Pivot rules

Rule	How it chooses	Sorted input	Random input	All-equal
First element	a[lo]	$O(n^2)$ — always the minimum	O(n log n)	$O(n^2)$
Random	random index in [lo, hi]	expected O(n log n)	expected O(n log n)	$O(n^2)$
Median of three	median of a[lo], a[mid], a[hi]	O(n log n) — picks the true middle	slightly fewer comparisons	$O(n^2)$

Median-of-three on sorted input picks the exact middle element every time, so it reproduced the perfect best-case count: **684,481** for n = 50,000 — the same as `theory_best(50000)`. On random data it was also the cheapest rule (800,600 vs ~909,000), because the median of three samples is closer to the true median than one random element.

On **reverse-sorted** input median-of-three needed 1,387,013 comparisons — still n log n scale ($n \log_2 n$ ≈ 780,482), just a larger constant: after the first partition the swaps leave the sub-arrays no longer neatly ordered, so later median-of-three picks are less central.

Median-of-three can still be forced into $O(n^2)$ by specially crafted "killer" inputs; only the random pivot gives an *expected* O(n log n) guarantee for every input.

PART 3

Multithreading, multiprocessing and the GIL

Why Quick Sort parallelises naturally

After one partition, the left part and the right part are **independent** — they occupy different index ranges and never touch each other. So they can be sorted at the same time by different workers, with no locks and no race conditions.

Your multithreaded version

```
THREADED-QUICKSORT(A, lo, hi, depth):
    if lo >= hi: return
    if depth == 0 or size < THRESHOLD (5000): sequential quicksort; return
    p = PARTITION(A, lo, hi)
    t = new Thread(THREADED-QUICKSORT(A, lo, p-1, depth-1))   # left part in a new thread
    THREADED-QUICKSORT(A, p+1, hi, depth-1)                    # right part in this thread
    t.join()                                                    # wait for the left part
```

- `depth` limits the number of simultaneous threads to **2^depth** (depth 1 → 2 threads, depth 3 → 8).
- `THRESHOLD = 5000` — creating a thread costs far more than sorting a few thousand elements, so small partitions are sorted sequentially. Without a limit, Quick Sort would try to create about n threads.
- `t.join()` makes the parent wait until the left part is sorted before returning — otherwise the caller could see a half-sorted array.

The Global Interpreter Lock (GIL)

In **CPython** (the standard Python), the GIL is a lock that allows only **one thread to execute Python bytecode at a time**. Threads still help for **I/O-bound** work (waiting on network or disk releases the GIL), but for **CPU-bound** work like sorting, two threads simply take turns on one core — plus switching overhead. That is why your threaded version got **0.83× – 1.01×**: never faster, sometimes slower.

(Python 3.13 introduced an experimental "free-threaded" build without the GIL — PEP 703 — but the standard build you used still has it.)

Your multiprocess version

Processes each have **their own Python interpreter and their own GIL**, so they truly run in parallel on different cores.

```
PROCESS-QUICKSORT(A, workers):
    segments = {(0, n-1)}
    while |segments| < 2 × workers: split the largest segment with PARTITION   # in the parent
    send a copy of each segment to a ProcessPoolExecutor with 'workers' processes
    each worker sorts its copy and returns it; the parent writes them back into A
```

Processes do **not share memory**, so every segment is **pickled (serialised), copied** to a worker and copied back. Making about 2 × workers segments keeps all workers busy even if partitions are uneven.

	Thread	Process
Memory	shared with other threads	separate address space
Creation cost	low	high (new interpreter)
Communication	direct (shared variables)	copying / pickling, pipes, queues
CPU parallelism in CPython	**no** (GIL)	**yes**
Good for	I/O-bound tasks	CPU-bound tasks

Why the speedup is below 2× and stops growing — Amdahl's law

AMDAHL'S LAW

Speedup(p) = 1 / (s + (1 − s) / p) where s = fraction of the work that stays serial, p = number of processors.
Even with infinite processors the speedup can never exceed 1 / s.

In your program the serial part is the **first partitions done in the parent** plus **copying data** to and from the workers. Your best speedup of 1.36× on 2 processes corresponds to an effective serial fraction of about 47% (1/1.36 = 0.47 + 0.53/2). Adding more workers than the **2 physical cores** cannot help — they just compete for the same cores and add more copying, so 4 and 8 workers fell to about 1.1×.

Concurrency vs parallelism: concurrency is *dealing with* many tasks at once (threads interleaving, as with the GIL); parallelism is *doing* many tasks at the same instant on different cores (your processes).

PART 4

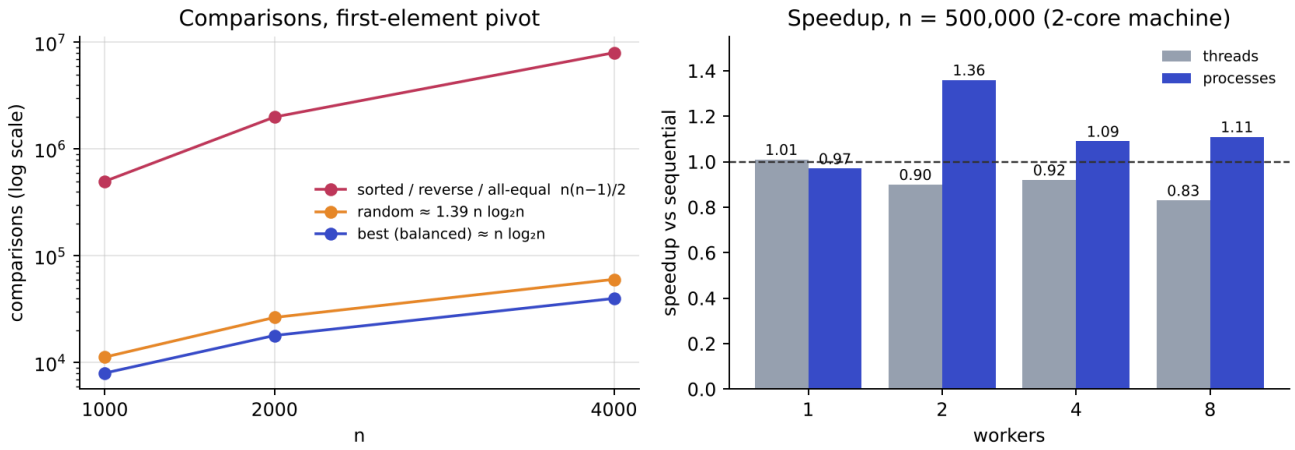
Code walkthrough — quicksort.py

Function	What it does
`choose_pivot(a, lo, hi, strategy)`	moves the chosen pivot to `a[lo]`: random index, or median of a[lo], a[mid], a[hi]; "first" does nothing
`partition(a, lo, hi, strategy)`	Lomuto partition with pivot at the front; exactly hi − lo comparisons; returns pivot's final index
`quicksort(a, lo, hi, strategy)`	sequential recursive Quick Sort, in place
`count_comparisons(data, strategy)`	sorts a copy and adds `hi - lo` per partition → exact comparison count (pivot-selection comparisons not counted)
`threaded_quicksort(a, lo, hi, depth)`	one new `threading.Thread` for the left part, current thread for the right, `join()` at the end; threshold and depth limits
`_sort_chunk(args)`	worker function run inside each process — sorts a chunk and returns it
`process_quicksort(a, workers)`	splits into ~2×workers segments in the parent, sorts them with `ProcessPoolExecutor`, writes results back
`gen_random / gen_sorted / gen_reverse / gen_equal / gen_best`	input generators; `gen_best` builds the perfectly balanced permutation
`theory_best / theory_worst / theory_random`	exact or expected comparison formulas (`lru_cache` memoises `theory_best`)
`time_ms(fn, data, runs)`	best-of-runs wall-clock time with `time.perf_counter()`; asserts the output is sorted
`exp1 … exp4`	the four experiments behind Tables 2–5
`main()`	`argparse` options `--quick` and `--workers`; `random.seed(0)` for repeatable runs

Good practices in your code to mention: every timing run sorts a **fresh copy** (`data[:]`) so later runs do not get already-sorted data; every result is checked with `is_sorted`; times are **best of 3** to reduce noise; `random.Random(seed)` makes inputs reproducible; the `if __name__ == "__main__":` guard is **required** for `ProcessPoolExecutor` on Windows/macOS (which start processes by re-importing the script — without the guard every child would start the benchmark again).

PART 5

Your results explained



Drawn from your Tables 2 and 5. Left: worst-case inputs need 50–200× more comparisons than the best case. Right: threads never beat sequential; processes peak at 2 workers = 2 cores.

Table 2 — best vs worst case (first-element pivot)

- **Doubling n roughly doubles best-case time** (1.4 → 3.2 ms) but **quadruples worst-case time** (66.2 → 266.5 ms). That is the practical signature of n log n vs n^2.
- At n = 4000 the worst case is **83× slower** than the best case and uses **200× more comparisons**.
- **Reverse-sorted is slower than sorted** (479.7 vs 266.5 ms) with the *same* number of comparisons: in reverse order every element is smaller than the pivot, so every comparison is followed by a **swap**. Comparisons are not the only cost.
- **All-equal is the fastest worst case** (259.5 ms): `<` is false for every element, so no swaps happen at all.

Table 3 — sequential vs threads vs processes (random input)

n	Threads	Processes	Reading
10,000	0.98×	**0.75×**	process start-up and copying cost more than the sort itself
100,000 – 1,000,000	0.90× – 1.00×	**1.28× – 1.31×**	real parallelism, limited by 2 cores and serial work

Table 4 — pivot rule (n = 50,000)

First-element on sorted input would need ≈ 1.25 billion comparisons (50,000 × 49,999 / 2), so it was skipped. Random and median-of-three handled sorted and reverse input with 0.7 – 1.4 million comparisons — close to $n \log_2 n$ = 780,482. **Pivot choice changes which inputs are bad, not the algorithm's worst-case bound** — except that random pivoting makes the bad case depend on luck rather than on the input.

Table 5 — speedup vs workers (n = 500,000)

Threads drift down from 1.01× to 0.83× as more threads add switching overhead. Processes peak at **1.36× with 2 workers** (= number of cores) and fall back to ~1.1× with 4 and 8 — more workers than cores only adds copying and scheduling.

Limitations and possible improvements

- Measured on a **2-core** machine: absolute times and speedups would differ on 8-core hardware, though the trends would not.
- Python is interpreted, so absolute times are much slower than C/C++/Java; threads in C++ or Java would give real speedup.
- **Three-way partitioning** (less / equal / greater, Dutch National Flag) would fix the all-equal case.
- **Shared memory** (`multiprocessing.shared_memory` or NumPy arrays) would avoid copying data to processes.
- **Insertion sort for tiny sub-arrays** (n < ~16) and **Introsort** (switch to Heap Sort when recursion is too deep) are standard production optimisations.

PART 6

Probable viva questions with answers

About the project

Q1. Explain your mini project in brief.

I implemented sequential, multithreaded and multiprocess Quick Sort in Python, compared their running times up to one million elements, counted comparisons to verify best-case n log n and worst-case n(n−1)/2, studied random and median-of-three pivots, and measured speedup against the number of threads and processes.

Q2. Why did you also implement a multiprocess version when the problem only asked for multithreaded?

Because in CPython the GIL lets only one thread run Python code at a time, so threads cannot speed up a CPU-bound sort. The multiprocess version shows what real parallel execution achieves, while the threaded version shows the design and its limitation.

Q3. Why did you count comparisons instead of only measuring time?

Comparison counts are exact and machine-independent, so they can be compared directly with the formulas n(n−1)/2 and the best-case recurrence. Time depends on the machine, other programs and the cost of swaps.

Q4. What inputs did you test and why each?

Random (average case), sorted and reverse-sorted (worst case for first-element pivot), all-equal (worst case for the strict `<` partition for every pivot rule), and a constructed best-case permutation (perfectly balanced partitions).

Q5. What hardware and software did you use? Why does it matter?

Python 3.13 on a 2-core Linux machine, standard library only. The number of cores sets the ceiling on parallel speedup — with 2 cores the ideal is 2×, and more than 2 workers cannot help.

Quick Sort theory

Q6. What is Quick Sort and which design strategy does it use?

A comparison-based, in-place sorting algorithm using divide and conquer: partition around a pivot so the pivot reaches its final position, then recursively sort both sides. There is no combine step.

Q7. What is the best, average and worst-case time complexity?

Best $\Theta(n \log n)$, average $\Theta(n \log n)$ ($\approx 1.39\, n \log_2 n$ comparisons), worst $\Theta(n^2)$ ($n(n-1)/2$ comparisons). Space $O(\log n)$ on average, $O(n)$ in the worst case.

Q8. When does the worst case occur in your implementation?

With the first-element pivot on sorted or reverse-sorted input (the pivot is always the min or max), and on all-equal input for every pivot rule, because the strict `<` puts every equal key on the same side.

Q9. Derive the worst-case number of comparisons.

Each partition of k elements makes k − 1 comparisons and removes only the pivot, so the total is (n−1) + (n−2) + ... + 1 = n(n−1)/2. For n = 4000 that is 7,998,000, exactly your measured value.

Q10. Derive the best-case complexity.

T(n) = 2T(n/2) + n. There are $\log_2 n$ levels and each does about n comparisons, so T(n) = Θ(n log n). By the Master theorem, f(n) = n = n^(log_b a), case 2.

Q11. How did you generate the best-case input?

By reversing the partition: for the median p and best-case arrangements L and R of the smaller and larger values, `[p] + L[1:] + [L[0]] + R` partitions into exactly L, p, R. Applied recursively, the first element of every sub-array is its median, e.g. [4, 1, 3, 2, 6, 5, 7] for 1...7.

Q12. Which partition scheme did you use? How many comparisons does it make?

Lomuto partition with the pivot at the front. It compares every other element once with the pivot, so exactly hi − lo comparisons for a sub-array a[lo..hi].

Q13. What is the difference between Lomuto and Hoare partition?

Lomuto uses one scanning index and a boundary; it is simpler but does more swaps. Hoare uses two indices moving towards each other and swapping out-of-place pairs; it does about three times fewer swaps on average and handles equal keys better (they are split evenly).

Q14. Why is reverse-sorted input slower than sorted input though both make the same comparisons?

In reverse order every element is smaller than the pivot, so every comparison triggers a swap. In sorted order no element is smaller than the first-element pivot, so no swaps are done. Swaps cost time too.

Q15. Why is all-equal input $O(n^2)$ even with a random or median-of-three pivot?

Every element equals the pivot, the test `a[j] < pivot` is always false, so all elements stay on one side. Each partition removes just one element whichever pivot is chosen.

Q16. How can the all-equal case be fixed?

Three-way partitioning (Dutch National Flag): split into < pivot, = pivot, > pivot and recurse only on the outer two parts. All-equal input then takes one O(n) pass. Hoare's partition also handles it in O(n log n).

Q17. Is Quick Sort stable? Is it in-place?

Not stable — long-distance swaps can reorder equal keys. In-place — it needs only the recursion stack, no extra array.

Q18. Why did you increase the recursion limit to 50,000?

Worst-case inputs recurse n levels deep. Python's default limit is about 1000, so sorting 4000 sorted elements would raise RecursionError.

Pivot rules

Q19. How does a random pivot change the worst case?

No fixed input can make it pick bad pivots consistently, so the expected time is O(n log n) for every input. The worst case $O(n^2)$ still exists but has negligible probability. It is a Las Vegas algorithm — always correct, random running time.

Q20. What is median-of-three? Why was it best on sorted input?

The pivot is the median of the first, middle and last elements. On sorted data the middle element is the true median of the sub-array, so every split is perfect — it reproduced the exact best-case count of 684,481 for n = 50,000.

Q21. Can median-of-three still hit $O(n^2)$?

Yes, on specially constructed "median-of-three killer" sequences, and on all-equal data with a two-way partition. Only randomisation removes the dependence on a particular input.

Q22. Which pivot rule would you use in practice?

Random or median-of-three combined with three-way partitioning, a cutoff to insertion sort for small sub-arrays, and a fallback to Heap Sort if recursion becomes too deep (Introsort, used by C++ std::sort).

Threads, processes and parallelism

Q23. What is a thread? What is a process?

A process is a running program with its own memory space. A thread is a lightweight unit of execution inside a process; threads of one process share its memory.

Q24. What is the GIL and how did it affect your results?

The Global Interpreter Lock in CPython lets only one thread execute Python bytecode at a time. For CPU-bound sorting the threads just take turns, so the threaded Quick Sort was never faster (0.83× – 1.01×) and got slower with more threads because of switching overhead.

Q25. Then why use threads in Python at all?

For I/O-bound work — downloading, reading files, waiting on databases — threads release the GIL while waiting, so many I/O operations overlap.

Q26. Why do processes give real speedup?

Each process has its own interpreter and its own GIL, so the operating system can run them simultaneously on different cores.

Q27. Why was the process version slower for n = 10,000?

Starting worker processes and pickling/copying the data to them and back takes longer than sorting 10,000 numbers sequentially. Parallelism pays off only when the work is large compared with the overhead.

Q28. What is THRESHOLD and why is it needed?

5000 — partitions smaller than this are sorted sequentially. Creating a thread or process has a fixed cost much larger than sorting a few thousand elements; without a threshold the program would create about n threads.

Q29. What does the depth parameter do?

It limits recursion levels that spawn threads, so at most 2^depth threads run at once. depth 1 → 2 threads, depth 3 → 8 threads.

Q30. Why is `t.join()` needed?

It waits for the child thread to finish sorting the left part. Without it the function could return while part of the array is still unsorted.

Q31. Do you need locks? Is there a race condition?

No. The two partitions cover disjoint index ranges, so the threads never read or write the same elements. A race condition happens only when threads access shared data concurrently and at least one writes.

Q32. Why didn't the speedup reach 2× on 2 cores?

The first partitions run serially in the parent, and data has to be copied to the worker processes and back. By Amdahl's law the serial part limits the speedup; 1.36× implies about 47% of the time was effectively serial.

Q33. Explain Amdahl's law.

Speedup with p processors = 1 / (s + (1 − s)/p), where s is the serial fraction. The serial part caps the speedup at 1/s no matter how many processors are added.

Q34. Why did speedup fall with 4 and 8 processes?

The machine has only 2 cores. Extra processes cannot run simultaneously; they only add start-up, copying and scheduling overhead.

Q35. Difference between concurrency and parallelism?

Concurrency: several tasks make progress by interleaving, possibly on one core (your threads under the GIL). Parallelism: tasks run at the same instant on different cores (your processes).

Q36. Why is the `if __name__ == "__main__":` guard important here?

On Windows and macOS, ProcessPoolExecutor starts each worker by importing the script. Without the guard every worker would run main() again and start its own benchmark — creating processes recursively.

Q37. How would the results differ in C++ or Java?

They have no GIL, so threads run in parallel and share memory without copying. The threaded version would get close to the number of cores in speedup for large n.

Code and measurement

Q38. How did you measure time accurately?

With `time.perf_counter()` (high-resolution clock), best of 3 runs for n ≤ 100,000, always on a fresh copy of the input, and with an `is_sorted` check after each run.

Q39. Why sort a copy (`data[:]`) every time?

Quick Sort sorts in place. Without a copy the second run would receive already-sorted data — the worst case for first-element pivot — and the timings would be wrong.

Q40. What is `lru_cache` used for?

It memoises `theory_best(n)`, so each sub-problem size is computed once — top-down dynamic programming for the recurrence.

Q41. Why were median-of-three's selection comparisons not counted?

`count_comparisons` counts only partition comparisons (hi − lo). The up-to-3 comparisons per call to choose the median were left out, as noted in the report; including them would add a small amount per partition.

Q42. What would you improve in this project?

Three-way partitioning, insertion sort for small sub-arrays, shared memory to avoid copying, testing on more cores, and trying Python's free-threaded build or a compiled language to see real thread speedup.